Daily Quant Digest: 2026-11-03

Foundations Restart, day 8. Three new steps today: how to find a probability by **splitting the world into cases** and adding them back up (the law of total probability, drawn as a tree), what a **bond** really is and why its price and its yield always move in opposite directions, and the **sliding window**, a way to scan a stream once instead of re-checking every stretch.

Today's topics

Track	Topic	Difficulty
Probability & Statistics	FR Lesson 8: the law of total probability: splitting into cases, tree diagrams	Easy-Medium
Derivatives	FR Lesson 8: bonds: coupons, price and yield, and why they move opposite	Easy-Medium
Algorithms & DS	FR Lesson 8: sliding window: best k-day P&L, then the longest stretch with no repeated symbol	Medium
Music	Fleetwood Mac, "The Chain" (1977)	

1. Probability & Statistics: Split Into Cases, Then Add Back Up

> **Where we are.** Lesson 2: events can be combined with "or", and if two events cannot happen together (disjoint), their probabilities simply add. Lesson 6: news **moves the walls in**, and $P(A \mid B)$ is the share of the small room B that A takes; the multiplication rule is $P(A \cap B) = P(B)\,P(A \mid B)$. Lesson 7: independence means the share does not move. Today we use conditional probabilities the other way round: not to shrink the room, but to **build up** a probability from pieces.

1.1 The central picture: water through pipes

Picture 1,000 litres of water poured in at the top of a set of pipes. The pipe splits into branches. At every split, the water divides in fixed shares: say 50% goes left, 30% goes down the middle, 20% goes right. Lower down, each branch splits again, and some of the water ends up in a bucket marked **"A"**.

How much water lands in bucket A? You cannot read it from one branch. You follow **every** branch, see how much water reaches A along it, and **add the amounts up**.

That is the law of total probability. The water is probability (1,000 litres = probability 1). Each split is a set of cases that are disjoint and cover everything.

In the picture	In the maths
All the water, 1,000 litres	The whole room, probability 1
The first split into branches	Cases $B_1, B_2, \ldots$: disjoint, and together they cover everything (a **partition**)
Share of water entering branch i	$P(B_i)$
Share of that branch's water that reaches bucket A	$P(A \mid B_i)$
Water reaching A through branch i	$P(B_i)\,P(A \mid B_i) = P(A \cap B_i)$
Total water in bucket A	$P(A)$: the sum over all branches

Where the analogy stops: water can leak. Probability cannot. If your branches do not cover everything, or two branches overlap (the same water counted twice), the totals come out wrong. That is exactly what the "partition" condition protects against.

1.2 Count first: 1,000 traders on three desks

A firm has 1,000 traders: 500 on FX, 300 on rates, 200 on equities. Last month the share of traders who lost money was 20% on FX, 10% on rates and 30% on equities. Pick a trader at random. What is the chance she lost money?

Follow the water, desk by desk:

Desk (branch)	Traders	Share who lost	Losers
FX	500	20%	100
Rates	300	10%	30
Equities	200	30%	60
Total	**1,000**		**190**

So $P(\text{lost}) = 190/1{,}000 = 19\%$.

Notice what we did. We never knew the overall loss rate. We knew it **per case**, and we knew how big each case was. Then we added up.

The tempting wrong answer. "The three rates are 20%, 10% and 30%, so the average is 20%." That treats the desks as equally big. They are not: FX has 500 people, equities only 200. The right answer is a **weighted** average, weighted by the size of each branch.

1.3 The tree diagram

The same calculation, drawn as a tree. Each branch is labelled with its probability. To get the chance of a path, **multiply along it** (Lesson 6's multiplication rule). To get the chance of an event, **add the paths** that end in it (Lesson 2's addition rule for disjoint events).

```
                          +-- lost      (0.2)   path: 0.5 x 0.2 = 0.10
          +-- FX    (0.5) -+
          |               +-- not lost  (0.8)   path: 0.5 x 0.8 = 0.40
          |
          |               +-- lost      (0.1)   path: 0.3 x 0.1 = 0.03
start ---+-- Rates (0.3) -+
          |               +-- not lost  (0.9)   path: 0.3 x 0.9 = 0.27
          |
          |               +-- lost      (0.3)   path: 0.2 x 0.3 = 0.06
          +-- Eq    (0.2) -+
                          +-- not lost  (0.7)   path: 0.2 x 0.7 = 0.14
                                                       six paths add to 1.00
```

$P(\text{lost}) = 0.10 + 0.03 + 0.06 = 0.19$. Same 19%.

Two quick checks you should always do on a tree: the branches leaving any point add to 1 ($0.5 + 0.3 + 0.2 = 1$, and $0.2 + 0.8 = 1$), and all the end paths add to 1.

1.4 The formula, one step at a time

In words: to find the chance of A, split the world into cases; in each case take "chance of the case" times "chance of A inside that case"; add these up.

$$P(A) = \sum_i P(B_i)\,P(A \mid B_i) = P(B_1)P(A \mid B_1) + P(B_2)P(A \mid B_2) + \cdots$$

What each symbol means. $B_1, B_2, \ldots$ are the cases (the desks). They must be disjoint (no trader on two desks) and must cover everyone (no trader on no desk). $P(B_i)$ is how big case i is. $P(A \mid B_i)$ is the share of case i where A happens. The $\sum_i$ sign means "add up over every case i".

Read aloud: "the probability of A is the sum, over all the cases, of the probability of the case times the probability of A given that case."

Why it is true, in three lines.

1. The pieces $A \cap B_1$, $A \cap B_2$, ... are the parts of A lying in each case. They do not overlap, and together they make up all of A (every world of A lies in exactly one case).
2. So by the addition rule, $P(A) = P(A \cap B_1) + P(A \cap B_2) + \cdots$.
3. By the multiplication rule, each piece is $P(A \cap B_i) = P(B_i)\,P(A \mid B_i)$. Substitute.

With numbers: $P(\text{lost}) = 0.5 \times 0.2 + 0.3 \times 0.1 + 0.2 \times 0.3 = 0.10 + 0.03 + 0.06 = 0.19$.

The simplest version: two cases. The cases are just "B" and "not B":

$$P(A) = P(B)\,P(A \mid B) + P(\text{not } B)\,P(A \mid \text{not } B).$$

A market example. Tomorrow the central bank either hikes rates (probability 0.3) or holds (0.7). If it hikes, the dollar rises against the yen with probability 0.8; if it holds, with probability 0.4. Then

$$P(\text{dollar rises}) = 0.3 \times 0.8 + 0.7 \times 0.4 = 0.24 + 0.28 = 0.52.$$

This is how a desk thinks about an event: list the scenarios, give each one a weight, ask what happens in each, and add up.

1.5 Common mistakes

- **Unweighted averaging.** As in 1.2: averaging 20%, 10% and 30% to get 20% ignores how big each case is.
- **Cases that overlap or leave gaps.** "Traders who use options" and "traders on the FX desk" overlap, so they cannot be your cases. Always check: does every world land in exactly one case?
- **Mixing up $P(A \mid B_i)$ and $P(B_i \mid A)$.** "20% of FX traders lost" is not "20% of losers are FX traders". In fact, of the 190 losers, 100 are FX: that is $100/190 \approx 53\%$. Turning the tree around like this is tomorrow's lesson (Bayes' rule).

1.6 Self-check

1. 60% of a desk's trades go through algorithm A, which makes a booking error 1% of the time; 40% go through algorithm B, which errs 3% of the time. What share of all trades have a booking error?
2. Draw two cards from a deck, without putting the first one back. What is the chance the **second** card is a heart?
3. In the tree of 1.3, what is $P(\text{not lost})$? Find it two ways.

1.7 Interview question: hidden cards, moved balls and dice-then-coins

(Warm-ups of this kind are common at SIG, Optiver and Jane Street. The interviewer wants to see you choose the right cases.)

a. A deck is shuffled and the top card is put aside face down, unseen. What is the probability that the **next** card is an ace?

b. Box 1 holds 3 red and 2 blue balls. Box 2 holds 1 red and 4 blue. You move one random ball from box 1 into box 2, then draw one random ball from box 2. What is the probability it is red?

c. You roll a fair die and then flip a fair coin as many times as the die shows. What is the probability that you see **no** heads at all?

1.8 Solution

(a) Cases: was the hidden card an ace?

- Hidden card is an ace: probability $4/52$. Then 3 aces remain among 51 cards: $3/51$.
- Hidden card is not an ace: probability $48/52$. Then 4 aces remain among 51: $4/51$.

$$P(\text{ace}) = \frac{4}{52}\cdot\frac{3}{51} + \frac{48}{52}\cdot\frac{4}{51} = \frac{12+192}{2{,}652} = \frac{204}{2{,}652} = \frac{1}{13}.$$

Exactly the same as if no card had been hidden.

Interview trick. By symmetry, every position in a shuffled deck is equally likely to hold any card, so the second card is an ace with probability $4/52 = 1/13$. A card you never look at cannot change your odds. (Self-check 2 is the same idea: $1/4$.)

(b) Cases: which colour was moved?

- Moved red (probability $3/5$): box 2 now has 2 red out of 6, so $2/6$.
- Moved blue (probability $2/5$): box 2 has 1 red out of 6, so $1/6$.

$$P(\text{red}) = \frac{3}{5}\cdot\frac{2}{6} + \frac{2}{5}\cdot\frac{1}{6} = \frac{6}{30} + \frac{2}{30} = \frac{8}{30} = \frac{4}{15} \approx 0.267.$$

(c) Cases: what did the die show? Each face $k = 1, \ldots, 6$ has probability $1/6$. Given k flips, all tails has probability $(1/2)^k$ (the flips are independent, Lesson 7). So

$$P(\text{no heads}) = \frac{1}{6}\left(\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \frac{1}{32} + \frac{1}{64}\right).$$

Inside the bracket, put everything over 64: $32 + 16 + 8 + 4 + 2 + 1 = 63$, so the bracket is $63/64$. Then

$$P(\text{no heads}) = \frac{1}{6}\cdot\frac{63}{64} = \frac{63}{384} = \frac{21}{128} \approx 0.164.$$

Sanity checks. (a) The answer must lie between $3/51$ and $4/51$ (it is a weighted average of the two), and $1/13 \approx 0.0769$ does. (b) $4/15 \approx 0.267$ lies between $1/6$ and $2/6$, and sits closer to $2/6$ because "moved red" is the more likely case. (c) With one flip, no heads would be $1/2$; with more flips it can only be smaller, so $0.164 < 0.5$ makes sense. All three fractions were also checked exactly by computer.

Self-check answers. (1) $0.6 \times 0.01 + 0.4 \times 0.03 = 0.006 + 0.012 = 0.018$, so 1.8%. (2) $\frac{13}{52}\cdot\frac{12}{51} + \frac{39}{52}\cdot\frac{13}{51} = \frac{156+507}{2{,}652} = \frac{663}{2{,}652} = \frac{1}{4}$. (3) Directly: $0.40 + 0.27 + 0.14 = 0.81$. By the complement: $1 - 0.19 = 0.81$.

---

2. Derivatives: Bonds, and the Seesaw Between Price and Yield

> **Where we are.** Lesson 1: money grows by compounding, $(1+r)^t$. Lesson 2: money later is worth less today; its present value uses a **discount factor**, $DF = 1/(1+r)^t$. Lesson 3: long gains when the price rises. Today we put these together into the most important product in finance: a bond is just a list of future payments, and its price is their present value.

2.1 The scene

The US government needs to borrow money. So does the Hong Kong airport authority, and so does Apple. Instead of going to a bank, they sell **bonds**: promises to pay a fixed set of amounts on fixed dates.

On the other side sits, for example, a Hong Kong pension fund. It must pay its members a steady income for decades, so it wants assets that pay steady, known amounts. A bond fits: the fund lends money today and receives regular **coupons** plus its money back at the end.

The key fact that makes bonds a trading product: once issued, the payments are **fixed**, but market interest rates keep moving. So the value of those fixed payments moves every day.

2.2 The central picture: a flat with a fixed rent

You own a flat that comes with a tenant on a **three-year lease at HKD 20,000 a month, fixed**. You want to sell the flat with the lease.

- If new leases in the same building are also HKD 20,000, a buyer pays you the normal price.
- If new leases now get **HKD 25,000**, your fixed HKD 20,000 looks poor. No buyer pays full price for it. You must **cut the price** until the deal is as attractive as a new lease.
- If new leases only get **HKD 15,000**, your old lease is a prize. Buyers pay **more** than normal.

The rent on your lease never changed. Only the market changed around it. That is a bond.

In the picture	In the bond
The fixed rent	The **coupon**, a fixed percentage of the face value
Getting the flat back at the end	The **face value** (or par), repaid at maturity
Length of the lease	**Maturity**
The going rent for new leases	The market **yield** for that maturity
What a buyer pays for flat plus lease	The bond's **price**
Market rent up, your price down	**Yields up, prices down**: the seesaw

Where the analogy stops: a real flat can rise in value by itself, and the tenant may stop paying. In today's bond the payments are fixed and certain (think of a US Treasury). Default risk is a later topic.

2.3 The mechanics, one at a time

Face value, coupon, maturity. Take a bond with face value USD 100, a **4% annual coupon** and **3 years** to maturity. It pays USD 4 at the end of years 1 and 2, and USD $4 + 100 = 104$ at the end of year 3. (US Treasuries pay half the coupon every six months; we use yearly payments to keep the arithmetic clean. The idea is identical.)

Price = present value of the payments. Each payment is discounted with Lesson 2's discount factor. If the market rate for this bond is y per year:

$$P = \frac{C}{1+y} + \frac{C}{(1+y)^2} + \cdots + \frac{C+F}{(1+y)^n}.$$

What each symbol means. P is the price today. C is the yearly coupon (USD 4). F is the face value (USD 100). n is the number of years (3). y is the **yield**. Each term is "a payment, shrunk by one factor of $(1+y)$ for every year we must wait for it".

Read aloud: "the price is the sum of each coupon divided by one plus the yield, raised to the number of years until it is paid, plus the face value discounted the same way."

What the yield is. In practice we see the **price** on a screen, and the yield is the one rate y that makes the formula give that price. It is called the **yield to maturity**: the yearly return you earn if you buy at this price, hold to the end, and everything is paid as promised. Traders quote and think in yields, because yields can be compared across bonds with different coupons and maturities, just as you compare flats by rent per square foot.

Why the seesaw is automatic. y sits in the **denominator** of every term. Raise y, every fraction gets smaller, so P falls. Lower y, P rises. There is no market opinion in this; it is arithmetic.

2.4 Worked example: the same bond at three yields

At $y = 4\%$ (yield equals coupon):

$$P = \frac{4}{1.04} + \frac{4}{1.04^2} + \frac{104}{1.04^3} = 3.8462 + 3.6982 + 92.4556 = 100.00.$$

When the coupon equals the yield, the bond is worth exactly its face value: it trades **at par**. (A new lease at the going rent sells at the normal price.)

At $y = 5\%$ (market rates rose by 1%):

$$P = \frac{4}{1.05} + \frac{4}{1.05^2} + \frac{104}{1.05^3} = 3.8095 + 3.6281 + 89.8391 = 97.28.$$

The coupon is below the yield, so the bond trades **at a discount** (below 100). The buyer gets 4 a year, less than the 5 a new bond would pay, and is compensated by paying only 97.28 now and receiving 100 at the end.

At $y = 3\%$:

$$P = 3.8835 + 3.7704 + 95.1747 = 102.83.$$

Coupon above yield: **at a premium**.

Yield	3-year 4% bond	10-year 4% bond
3%	102.83	108.53
4%	100.00	100.00
5%	97.28	92.28

Read the table slowly. Three lessons hide in it.

1. **Seesaw.** Up yield, down price, in every column.
2. **Longer bonds swing more.** A 1% rise costs the 3-year bond 2.72 but the 10-year bond 7.72. A lease with ten years left is stuck with the poor rent for much longer. (Measuring this is Lesson 9: duration.)
3. **Not symmetric.** For the 10-year bond, a 1% fall in yields gains 8.53 but a 1% rise loses only 7.72. The price curve bends. This bend is called **convexity**, and we will meet it again.

The trade. A pension fund buys USD 10 million face of the 3-year 4% bond at par, paying USD 10,000,000. The next week, yields for 3 years jump to 5%. The bond is now worth $97.2768\% \times 10{,}000{,}000 = \text{USD } 9{,}727{,}680$. The fund shows a mark-to-market loss of **USD 272,320**, although the bond issuer has done nothing wrong and every payment is still certain.

If the fund simply holds to maturity, it still receives USD 400,000 a year and USD 10 million at the end. With no further moves, the price climbs back towards 100 as maturity approaches (98.14 with two years left, 99.05 with one year left): this is the **pull to par**. The loss was real only for someone who has to **sell** before then.

2.5 When the seesaw broke a bank

Silicon Valley Bank, 2023. In 2020 and 2021 SVB received a flood of deposits and invested much of them in long-dated US Treasuries and mortgage bonds, at very low yields. In 2022 the Federal Reserve raised rates fast, yields jumped, and those bonds fell sharply in price, with unrealized losses of roughly USD 15 billion on the bonds it had labelled "held to maturity" alone, around the size of the bank's entire equity. The bonds were safe from default; the problem was the seesaw. When depositors started withdrawing, SVB had to sell bonds at a loss (about USD 1.8 billion announced on 8 March 2023), which caused panic and a bank run. Regulators closed it on 10 March 2023. Like Metallgesellschaft in Lesson 7: being right in the long run does not help if you are forced to sell in the short run.

2.6 Likely interview questions

Q1. Why do bond prices fall when yields rise? The payments are fixed. The price is their present value, and the yield is the discount rate in the denominator, so a higher yield shrinks every term. In plain words: a new bond now pays more, so nobody pays full price for an old bond paying less.

Q2. A bond's coupon is 6% and its yield is 5%. Premium or discount? What happens to its price over time if yields stay put? Premium (coupon above yield). For 3 years with annual coupons the price is 102.72. If yields stay at 5% it drifts **down** to 100 at maturity: pull to par.

Q3. Price a 2-year zero-coupon bond with face 100 at a 5% yield. One payment, so $P = 100/1.05^2 = 90.70$. A zero is the cleanest bond: its price is just face times a discount factor.

---

3. Algorithms & Data Structures: The Sliding Window

> **Where we are.** Lesson 1: Big-O. Lesson 4: hash maps and sets, and counting with a table. Lesson 5: prefix sums. Lesson 7: two pointers, where we threw away elements that **provably cannot help**. Yesterday's last note said two pointers can also walk in the **same** direction. Today that becomes the sliding window: a left edge and a right edge that both only ever move right.

3.1 The central picture: a train window

You sit on a train and look through a window. The scenery slides past. At each moment you see a stretch of the landscape: one new tree enters on one side as an old one leaves on the other. You never have to look at the whole landscape again from scratch; you only notice what came in and what went out.

A **sliding window** is a stretch of an array or string, from index `left` to index `right`. We move it along and keep a small **summary** of what is inside (a sum, a count table), updating the summary only for the element that enters and the element that leaves.

In the picture	In the code
The landscape	The array or string
The edges of the window	Indices `left` and `right`
What you can see	Elements `left..right`, length `right - left + 1`
A tree enters	`right` moves on, add `s[right]` to the summary
A tree leaves	`left` moves on, remove `s[left]` from the summary
Remembering the best view	Updating `best`

There are two kinds of window. A **fixed** window always has the same length (a train window). A **variable** window grows on the right and shrinks on the left as needed (a window you can stretch). Where the analogy stops: a train only moves forward, and so do our edges; that is the whole point. If an edge ever had to move backwards, the method would lose its speed.

3.2 Warm-up: the best 3-day P&L

Daily P&L: $[4, -2, 7, 1, -5, 6, 3, -1]$. Which 3 consecutive days made the most?

Brute force: for each start, add up 3 numbers. With window length k and n days that is about $n \times k$ steps.

Window: add the first 3 days: $4 - 2 + 7 = 9$. Then slide: each step add the day entering and subtract the day leaving.

Window (days)	Enters	Leaves	Sum
0 to 2			9
1 to 3	1	4	6
2 to 4	-5	-2	3
3 to 5	6	7	2
4 to 6	3	1	4
5 to 7	-1	-5	8

Best: **9**, days 0 to 2. Two operations per step, so $O(n)$ whatever k is.

3.3 The problem

A market data feed sends a stream of single-character venue codes, one per message (in general, any bytes). Find the length of the **longest stretch of consecutive messages in which no code repeats**. (This is the classic interview problem “longest substring without

repeating characters”.)

- Input: a `string s`, $0 \le n \le 5 \times 10^4$, any characters.
- Output: the maximum length of a substring with all characters distinct.

Input	Output	A longest stretch
`"abcabcbb"`	3	`"abc"`
`"bbbbb"`	1	`"b"`
`"pwwkew"`	3	`"wke"` (`"pwke"` is not consecutive)
`""`	0	
`"abba"`	2	`"ab"` or `"ba"`
`"ELHTLEHK"`	5	`"TLEHK"`

3.4 Brute force first

Try every start `i`. Walk `j` to the right, marking characters as seen, and stop at the first repeat. Record the length. That is up to n starts, each walking up to n steps: $O(n^2)$. For $n = 5 \times 10^4$ it is up to about 2.5×10^9 steps in the worst case (in practice the walk stops after at most 256 steps, but we want a method that does not rely on that). We keep it as the **checker** in the test code.

The waste: after starting at `i` and finding a repeat at `j`, we restart from `i + 1` and re-check almost the same characters again.

3.5 The insight: a window that stretches and shrinks

Keep a window that is always valid (no repeats inside). Move `right` one step at a time:

1. **Grow:** take `s[right]` into the window.
2. **Repair:** if that character is now in the window twice, move `left` forward, dropping characters, until the old copy has left. Now the window is valid again.
3. **Record:** the window `left..right` is the longest valid stretch **ending at** `right`. Update `best`.

Why is it safe never to move `left` back? Same argument as yesterday’s discarding. Suppose the window `left..right` contains a repeat. Then every **wider** window that starts further left and ends at `right` contains that same repeat. So no valid stretch ending at `right` (or later) can start before the repaired `left`. Characters we drop are never needed again.

Walk through `"pwwkew"` (indices 0 to 5):

right	char	window after repair	length	best
0	p	`p`	1	1
1	w	`pw`	2	2
2	w	repeat: drop `p`, drop `w`, window `w`	1	2
3	k	`wk`	2	2
4	e	`wke`	3	3
5	w	repeat: drop `w`, window `kew`	3	3

Complexity. `right` moves n times. `left` also moves at most n times **in total**, because it only moves forward. So the whole scan does at most $2n$ moves: $O(n)$ time. The summary is a table of 256 counters, one per possible byte: $O(1)$ extra space (it does not grow with n).

A faster repair: jump instead of walk. Instead of counts, remember `last[c]`, the index where character `c` was last seen. When `s[right]` was last seen at index `p` **inside** the window ($p \geq$ `left`), jump `left` straight to `p + 1`. One step instead of a walk.

The trap is `"abba"`. At the final `a`, its old copy is at index 0, but `left` is already at 2 (it jumped there when the second `b` arrived). If we blindly set `left = 0 + 1`, the window would move **backwards** and contain `bba`, which has a repeat, giving the wrong answer 3. The test `last[c] >= left` prevents this: an old copy that is already outside the window is ignored.

3.6 C++ solution

```cpp
// Daily Quant Digest 2026-11-03
// FR Lesson 8: sliding window.
// Warm-up: best total P&L over any k consecutive days (fixed-size window).
// Problem: longest stretch of a feed with no repeated symbol
//          (classic "longest substring without repeating characters").
// Compile: g++ -std=c++17 -O2 -Wall 2026-11-03.cpp -o window && ./window
#include <algorithm>
#include <array>
#include <string>
#include <vector>

using std::array;
using std::string;
using std::vector;

// Warm-up: the largest sum of k consecutive entries. Requires 1 <= k <= pnl.size().
long long bestKDaySum(const vector<int>& pnl, int k) {
    long long window = 0;
    for (int i = 0; i < k; ++i) window += pnl[i];       // first window: days 0..k-1
    long long best = window;
    for (int right = k; right < (int)pnl.size(); ++right) {
        window += pnl[right];          // the new day walks in on the right
        window -= pnl[right - k];      // the oldest day walks out on the left
        best = std::max(best, window);
    }
    return best;
}

// Version 1: the general "grow right, shrink left" template with a count table.
```

```cpp
int longestUniqueShrink(const string& s) {
    array<int, 256> count{};          // count[c] = copies of byte c inside the window
    int best = 0, left = 0;
    for (int right = 0; right < (int)s.size(); ++right) {
        unsigned char c = s[right];
        ++count[c];                                     // grow: take s[right] in
        while (count[c] > 1) {                          // window broken: c twice
            --count[(unsigned char)s[left]];            // shrink: drop s[left]
            ++left;
        }
        best = std::max(best, right - left + 1);        // window is valid here
    }
    return best;
}

// Version 2: jump the left edge directly, using where each symbol was last seen.
int longestUniqueJump(const string& s) {
    array<int, 256> last;
    last.fill(-1);                    // -1 = "not seen yet"
    int best = 0, left = 0;
    for (int right = 0; right < (int)s.size(); ++right) {
        unsigned char c = s[right];
        if (last[c] >= left)          // c is already inside the window
            left = last[c] + 1;       // jump just past its old copy (never backwards)
        last[c] = right;
        best = std::max(best, right - left + 1);
    }
    return best;
}
```

The full file `2026-11-03.cpp` also has a brute-force checker, all the examples above, and a randomized test: 20,000 random strings over small alphabets (so repeats are frequent), where both versions must match the brute force, plus random arrays for the warm-up. All pass.

3.7 Walkthrough of the C++

`const vector<int>& pnl`. A reference to the caller's vector (no copy), marked `const` so the function promises not to change it. For big inputs this matters: passing by value would copy 50,000 numbers on every call.

`long long window`. A sum of many `int`s can overflow a 32-bit `int` (about $\pm 2.1 \times 10^9$). `long long` is at least 64 bits. Good habit for any running total.

`(int)pnl.size()`. `size()` returns an **unsigned** type (`size_t`). Comparing a signed `int` with an unsigned value gives a warning under `-Wall`, and worse, with a negative number on the signed side the comparison silently goes wrong. Casting once to `int` keeps all the loop arithmetic signed. It is safe here because n is far below the `int` limit.

`std::max(best, window)`. From `<algorithm>`. Both arguments must have the **same type**: `std::max(best, right - left + 1)` works because both are `int`; mixing `long long` and `int` would not compile, a common surprise.

`array<int, 256> count{};`. `std::array` is a fixed-size array whose size is part of its type. It lives on the stack (no heap allocation), knows its size, and can be copied like a value. The `{}` is important: it **zero-initializes** all 256 counters. Without it, a local array of `int`s holds

garbage.

Why 256 and `unsigned char`? A `char` holds one byte, but on most compilers plain `char` is **signed**: bytes above 127 (for example parts of UTF-8 accented letters) come out as **negative** numbers, and `count[-61]` would read memory outside the array (undefined behaviour). Converting to `unsigned char` maps every byte to 0..255, a valid index. The line `unsigned char c = s[right];` does this conversion; in `--count[(unsigned char)s[left]]` we do it with a cast. The test file checks a UTF-8 string to make sure.

`last.fill(-1);`. `fill` is a member function of `std::array` that sets every element. We need -1 (not 0) as "never seen", because index 0 is a real position.

`++count[c]` vs `count[c]++`. Both add one. The prefix form `++x` is the usual C++ style when you do not need the old value.

`while (count[c] > 1)`. This inner loop looks like it could make the function $O(n^2)$, but it cannot: each pass moves `left` one step forward, and `left` never exceeds `right`. Over the whole function, the inner loop runs at most n times **in total**. This "amortized" argument (count the total, not per outer step) is the key to every sliding window.

`if (last[c] >= left) left = last[c] + 1;`. The guard from the `"abba"` trap. Equivalently you could write `left = std::max(left, last[c] + 1);`, which makes "never move backwards" explicit.

Which version to use? In an interview, write version 1 first: the grow / repair / record pattern works for many window problems (at most k distinct symbols, sum at most a budget, and so on). Then mention version 2 as an optimization for this specific problem.

Self-check. (1) Why does the warm-up not need a count table? (2) What does version 2 return for `"tmmzuxt"`, and which line protects it? (3) Could a sliding window find the longest stretch with total P&L **exactly** k when P&Ls can be negative?

Answers. (1) A sum can be updated by adding and subtracting; no other information is needed. (2) 5 (`"mzuxt"`); at the final `t`, its old copy at index 0 is before `left = 2`, and the `>= left` test ignores it. (3) Not safely: with negative numbers, extending the window can make the sum go down **or** up, so "shrink when too big" is no longer a safe rule. That is why Lesson 5 used prefix sums and a hash map for it.

---

4. Music: Fleetwood Mac, "The Chain" (1977)

Context

"The Chain" is on **Rumours** (February 1977), one of the best-selling albums of all time. It is the only song on the album credited to all five members of the classic line-up: **Lindsey Buckingham**, **Stevie Nicks**, **Christine McVie**, **John McVie** and **Mick Fleetwood**. It

was not one of the album's hit singles, yet it has become one of the band's signature songs.

The story behind it

Rumours was made while the band was falling apart. John and Christine McVie were divorcing, Lindsey Buckingham and Stevie Nicks had split up after years together, and Mick Fleetwood's own marriage was ending. They kept working together anyway, in the studio every day with the people they had just broken up with.

"The Chain" was not written in one go. It was assembled in the studio from separate pieces: parts of earlier unfinished songs by Buckingham and Christine McVie, a new section built on John McVie's bass line, and words that Nicks wrote. The tape was literally cut and spliced to join the parts. So the song about a bond that will not break is itself stitched together from the work of five people who could barely stand each other at the time. In Britain, the instrumental ending became famous on its own as the theme of the BBC's Formula One coverage.

Why people love it

It is dark and tense in a way the rest of Rumours mostly is not. The three voices of Buckingham, Nicks and Christine McVie sing together, but the harmony sounds hard and defiant rather than sweet. Listeners hear the real story underneath: people who are hurt, but who refuse to break up the band.

What to listen for

- **The start.** A sparse, stomping kick drum and a dark, picked acoustic guitar figure from Buckingham. It feels almost like a folk song or a march.
- **The three-part harmony** on the chorus: listen for each voice separately, then all three together.
- **The silence before the bass.** About three minutes in, the song nearly stops. Then John McVie's bass line enters alone: a simple pattern of just a few notes. This is the part everyone recognises.
- **The build.** Fleetwood's drums come in behind the bass, then Buckingham's sharp, aggressive electric guitar solo takes over and drives the song to its end. Notice how the whole second half is built on that one bass line.